



Γειά 喂 สวัสดี hello
안녕하세요 olá Oii Olá! ciao hola
Hej Привет guten tag
aloha goddag Chào ahn / Chào chị
今日は こんにちは shalom
hallo Привет

CMPU4003: Advanced Databases

- TU856, TU857, TU858
- Dr. Deirdre Lawless
 - e: deirdre.lawless@tudublin.ie



Öngietorri Akwaba Dobrodosli! Üdvözlét! Yokoso Accogliere! Mai Mai Tem-vindo Willkommen
Benvido! Haere Mai Salve
Degemer Tervetuloa Accoje Hosgeldiniz Benvignuo Fäilite Swagata Welkom Witaj Bienvenue
Nayak Chao nung Vithaykham Bi xê lai Nayak Chao nung Vithaykham Bi xê lai

Contents



What is Advanced Databases about?



Indicative Syllabus



Learning Outcomes



Assessments



Reading List

What do you know from Databases 1 and Databases 2?

You have already worked with:

- relational modelling;
- SQL;
- keys and constraints;
- transactions and concurrency;
- **ACID v BASE**;
- indexes;
- EXPLAIN ANALYZE;
- NoSQL concepts.

We are not starting again. CMPU4003 builds on these concepts.



Join at:
vevox.app

ID:
169-396-804



How comfortable are you
with your knowledge and
skills?

<https://vevox.app/#/m/169396804>

169-396-804

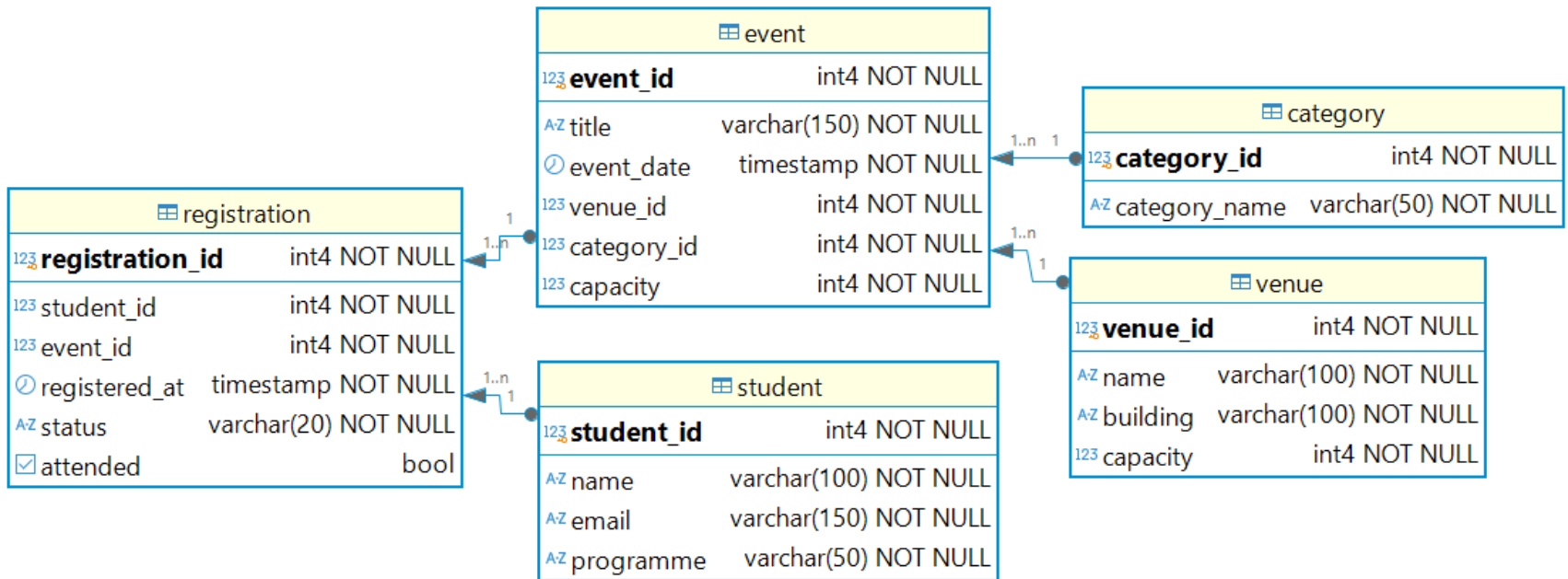
A screenshot of the Vevox Audience Engagement web interface. The header is dark blue with the ID "169-396-804" and a home icon. The main content area has a light blue background with the Vevox logo and the text "Audience Engagement". It features illustrations of a hand holding a smartphone displaying a survey, another hand holding a smaller smartphone, and a laptop showing a person's profile. A red circle highlights a notification icon in the top right corner.

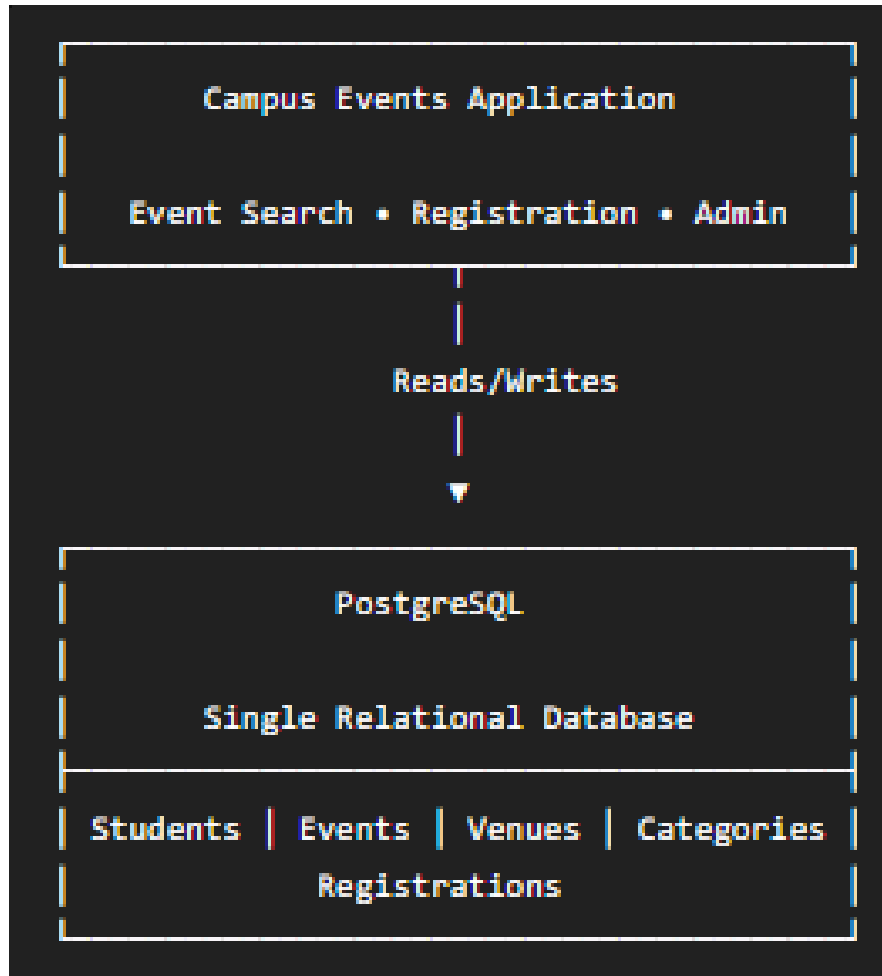
CMPU4003 Advanced Databases Week1

Our Case Study-

Campus Events system as a common case study.

The system allows students to discover and register for events taking place across the university, including academic, careers, social, wellbeing and sporting events.

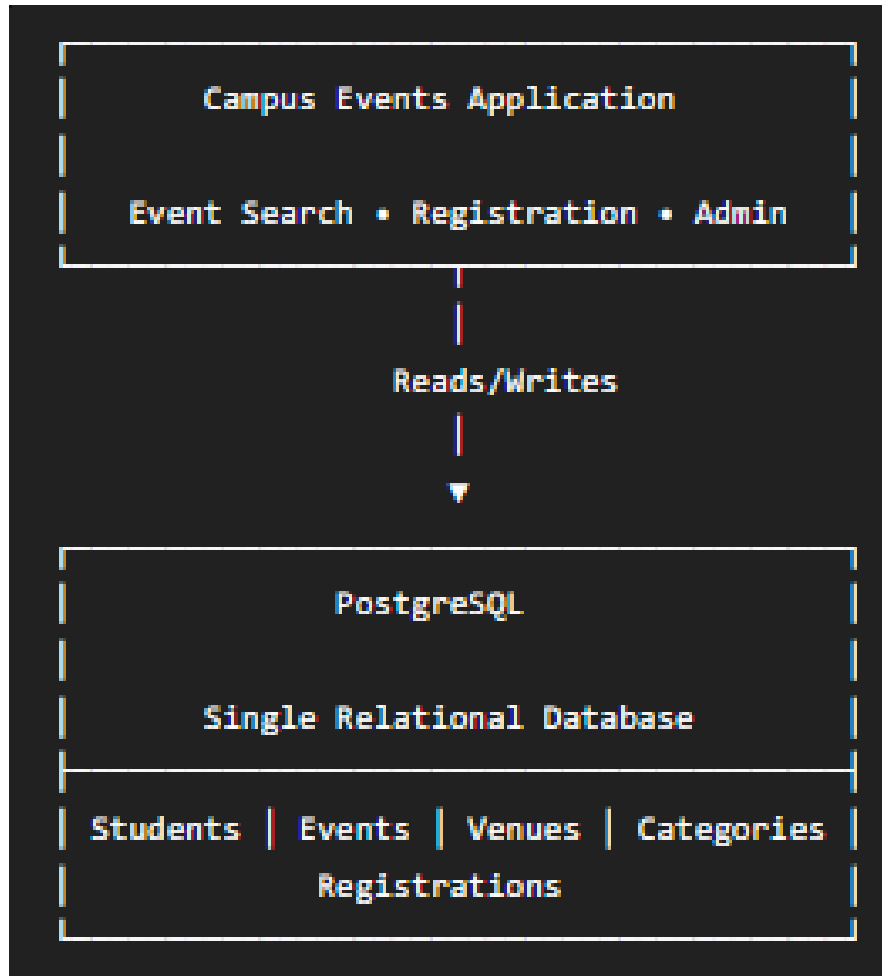




Initial Architecture

Is there anything wrong with this architecture?

Probably Not



Initial Architecture

Data is normalised;

Constraints protect integrity;

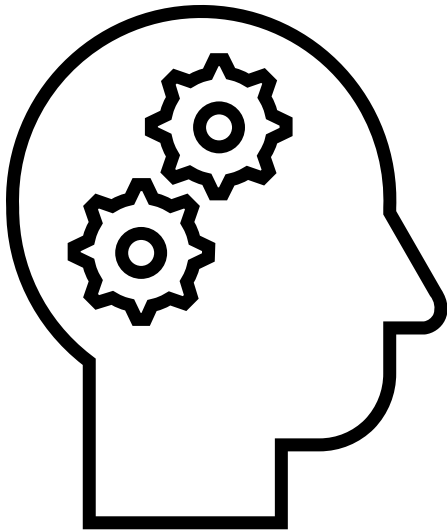
SQL answers the questions;

transactions can handle
concurrency;
indexes/query optimisation can
improve performance.

So why do we need an Advanced Databases module?

What happens when the requirements change?

Problem 1: The data changes



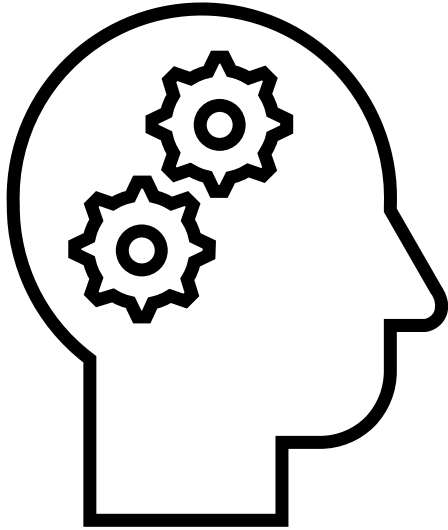
Originally every event looks broadly similar:

- Event (title, date, venue, category, capacity)

Now we want to accommodate:

- Academic Event (speaker, abstract, research_area)
- Sports Event (team_size, equipment, changing_facilities)
- Careers Event (employers, job_categories, registration_url)

Do we keep adding columns and tables?



Problem 2: The questions change

Operational system: Is there a place available at tonight's event?

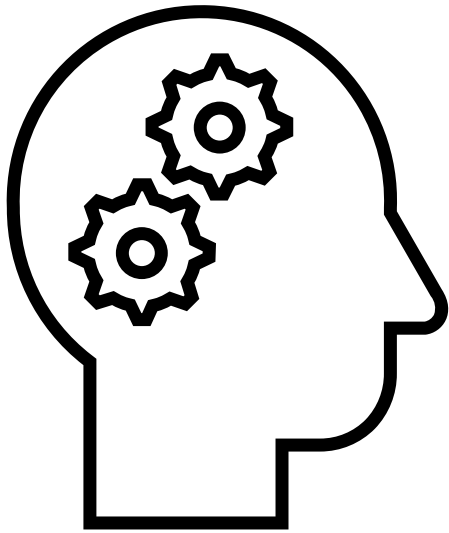
Management: How has attendance varied by programme, category, venue, semester and year over the last five years?

Operational (Online Transaction Processing - OLTP)	Analytical (Online Analytical Processing - OLAP)
Current data	Historical data
Small number of rows	Potentially millions
Frequent updates	Mostly analysis
Fast individual transactions	Aggregation across large datasets
Registration	Trends

OLTP ≠ OLAP

Data Warehouse / Dimensional Model

Problem 3: The system gets big



Suppose Campus Events grows to have:

100,000 students

50,000 events

10 million registrations

Can PostgreSQL store 10 million registrations?

The question isn't simply **“Can it store it?”**

The question is **“How should the data be physically organised for the workload?”**

What do we mean by workload?

A DATABASE WORKLOAD DESCRIBES HOW THE DATABASE IS BEING USED.

DIFFERENT WORKLOADS PLACE DIFFERENT DEMANDS ON THE DATABASE.

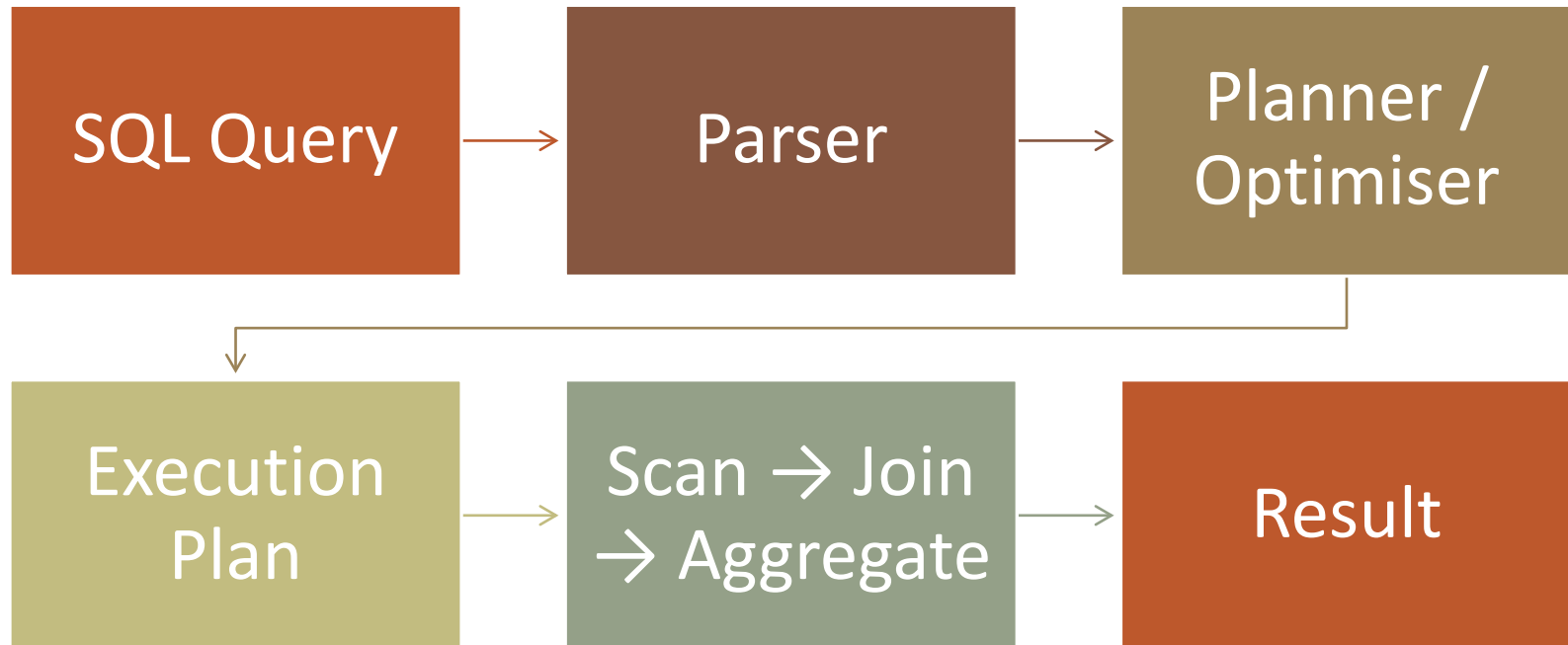
Campus Events requirement	Workload characteristic
Find an event by ID	Small, selective lookup
Find upcoming Technology events	Filter + range query
Register a student for an event	Transactional read/write
Count attendance by programme for five years	Large analytical aggregation
Increment an event's view count	Frequent small updates
Find events similar to a student's interests	Similarity/search

Logical Design	Physical Design
What data is stored?	How is it accessed efficiently?
Tables/entities	Indexes
Attributes/columns	Access paths
Relationships	Storage organisation
Primary/foreign keys	Partitioning
Integrity constraints	Physical organisation for workload
Driven mainly by data requirements	Driven heavily by workload

The logical model may remain correct while the physical design needs to change.

Logical v Physical Design

From SQL to Result



PostgreSQL considers alternative ways of executing a query and selects a plan based on estimated cost.

EXPLAIN (ANALYZE, BUFFERS)

SELECT ...

Don't guess whether a change improves performance — measure it.

How Do We Know What PostgreSQL Did?

Will an Index
Make This
Query Faster?

```
SELECT *  
FROM registration  
WHERE student_id = 1042;
```

Will an Index
Make This
Query Faster?

```
EXPLAIN (ANALYZE, BUFFERS)
```

```
SELECT *
```

```
FROM registration
```

```
WHERE student_id = 1042;
```

```
CREATE INDEX idx_registration_student ON  
registration(student_id);
```

```
EXPLAIN (ANALYZE, BUFFERS)
```

```
SELECT *
```

```
FROM registration
```

```
WHERE student_id = 1042;
```


Will an Index Make This Query Faster?

No.

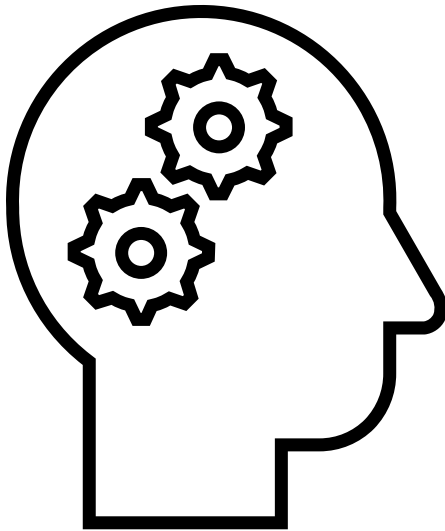
PostgreSQL's optimiser decides whether using the index is expected to be cheaper than alternatives.

Creating an index and demonstrating that an index improves a workload are different things.

Don't assume an index will improve performance.

Examine the workload, measure the current behaviour, make a physical-design change, and measure again.

Problem 4: One server isn't enough

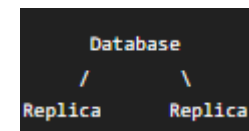


New requirements:

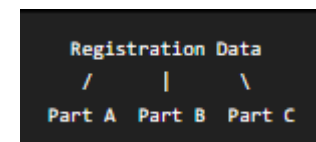
- more users;
- more data;
- high availability;
- server failure;
- perhaps geographically distributed users.

What could we do?

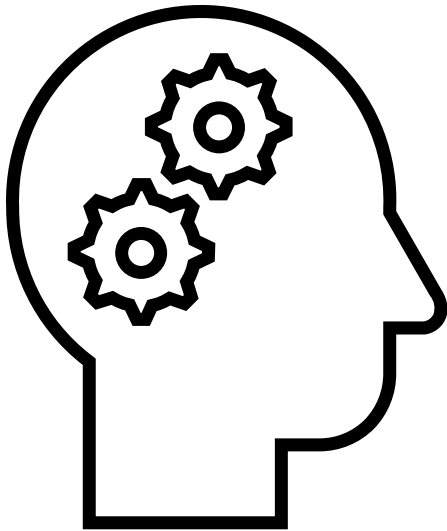
Replication



Partitioning



Problem 4: One server isn't enough



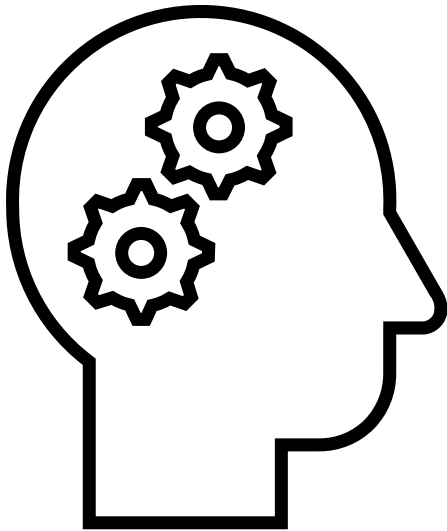
New requirements:

- more users;
- more data;
- high availability;
- server failure;
- perhaps geographically distributed users.

Distributed databases

What happens if the network fails but the database nodes remain operational?

Problem 5: The query itself changes



Suppose a student searches:

“Events that would help me prepare for a career in AI.”

Traditional relational search might look for:

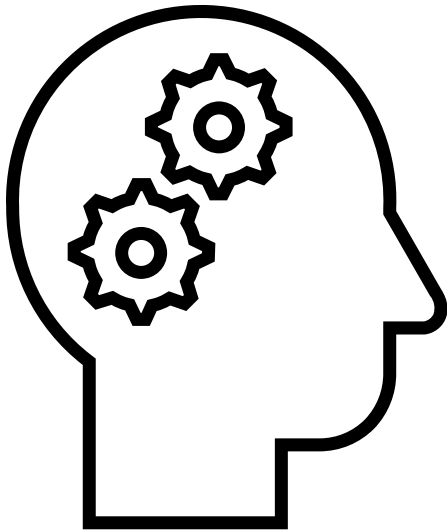
- WHERE title LIKE '%AI%'

But that might miss:

- Machine Learning Careers;
- Working as a Data Scientist;
- Graduate Opportunities in Artificial Intelligence;
- Careers in Applied Analytics.

What if we want to search by meaning rather than exact values or keywords?

Problem 5: The query itself changes



Embeddings and Vector Search

Event description



Embedding



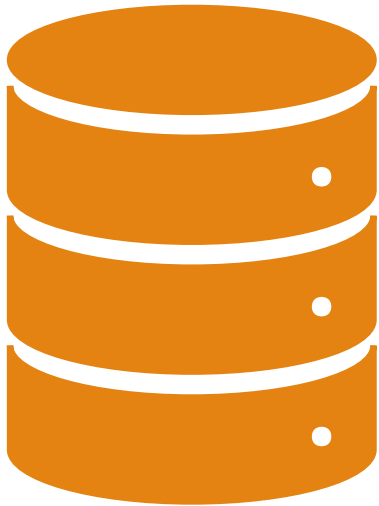
Vector



Similarity Search

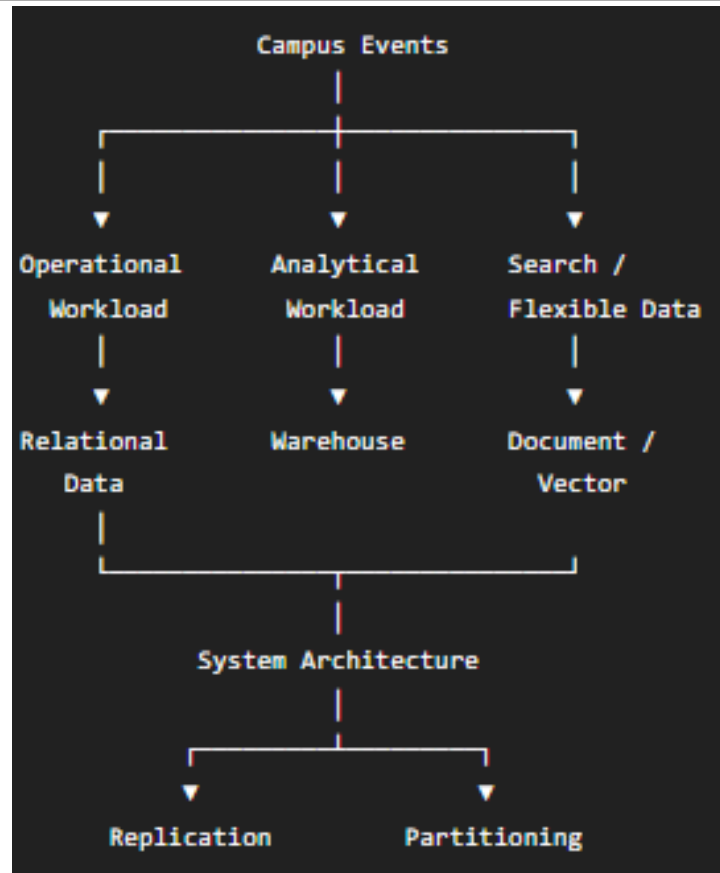
This gives us another type of database workload to consider.

Does one database do everything?



Polyglot persistence

Different parts of an application may use different data models or database technologies because they have different requirements.





PostgreSQL can do most things we want for our Campus Events data.

So why would we ever need anything else?

By the end of this module, you should be able to answer that question — and justify your answer in terms of **data, workload, performance, consistency, availability and architecture**.

Consider	Questions
Data	What does it look like? How does it change?
Workload	Reads? Writes? Analytics? Search?
Scale	How much data? How many users?
Performance	What response times are required?
Consistency	How current/correct must every copy be?
Availability	What happens when something fails?
Distribution	Where is the data and processing?
Integration	How does data move between systems?

There is no best choice

Don't ask: Which database is best?

Ask: **Which approach is appropriate for this workload?**

Concept / Approach	Technology We'll Use
Relational	PostgreSQL
JSON / Document	PostgreSQL / Redis
Key-value / In-memory	Redis
Vector Search	Redis / PostgreSQL, as appropriate
Data Warehouse	PostgreSQL
Partitioning	PostgreSQL
Replication / Distribution	PostgreSQL / Redis

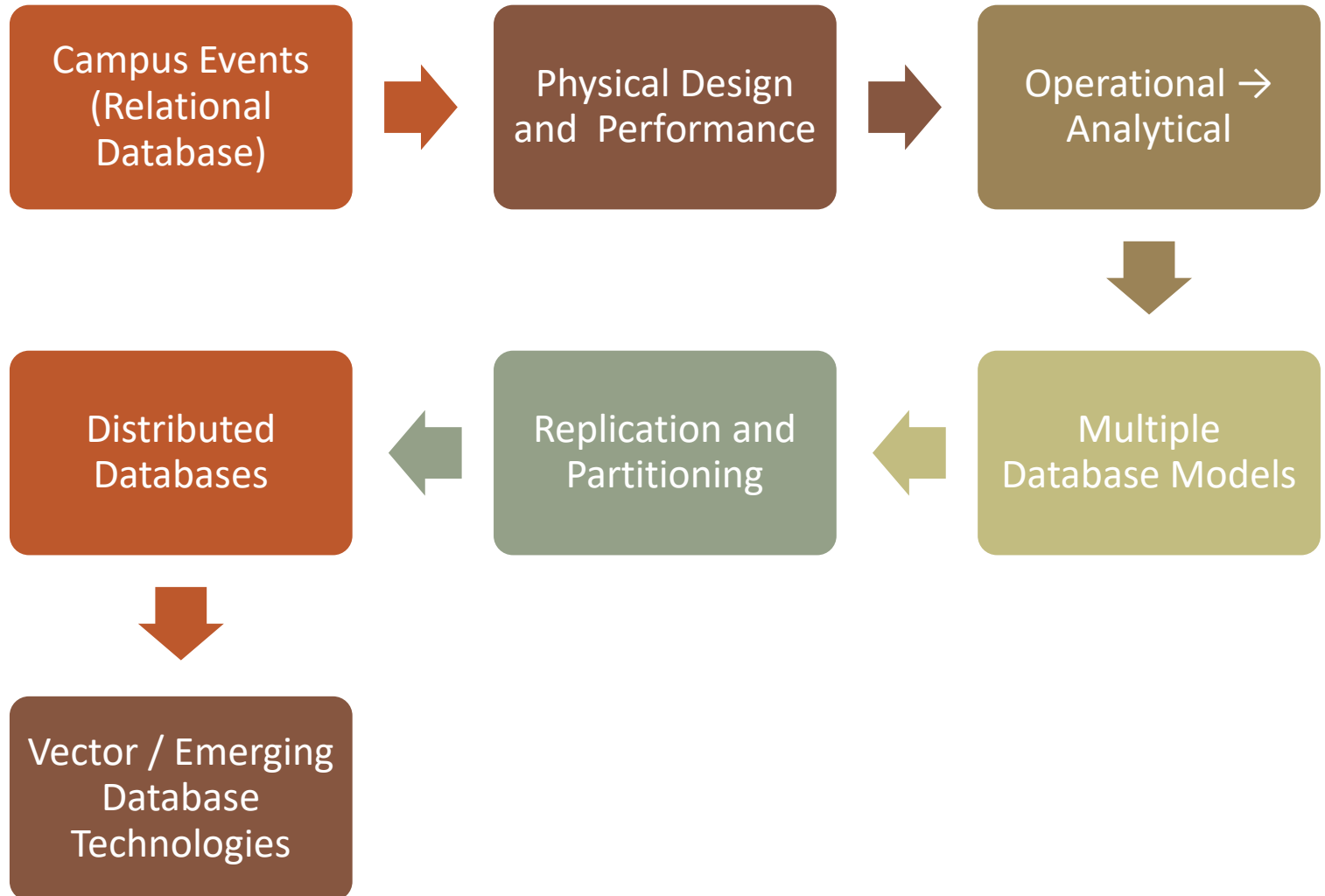
Our Database Environment

PostgreSQL and Redis are our tools.

The module is not a PostgreSQL and Redis training course.

The learning is about the **underlying models, architecture and trade-offs.**

How you will work



Next Week: Campus Events Gets Bigger

You will receive a much larger Campus Events database and a set of application workloads.

The question is no longer simply:

Can you write the SQL?

The question becomes:

How should the database be physically designed for this workload, and what evidence supports your decision?

Measure → Examine → Change → Measure again.

Module Overview

This module provides knowledge and practical application of advanced database concepts, building on the experience and knowledge gained on core database modules from previous years.

The module covers advanced and complete database concepts relating to storage, management and architecture of data. It also covers topics in advanced database design, focussing on logical, physical design aspects and dimensional modelling.

Emerging database technologies and themes will also be studied, including NoSQL DBs and graph databases.

Module Aims

The aim of this module is to study and practice advanced database features and techniques for relational and other database models; and to understand alternate database architectures in the context of applications and information systems.



Indicative Syllabus

1. Relational database design

- Reviewing and distinguishing between conceptual and logical design and addressing considerations and implications of implementing physical design from logical design.

2. Distributed databases

- Definitions, rationale, architectures, design, implementation, and manipulation; approaches to fragmentation and replication (synchronous and asynchronous); the CAP theorem.

3. Data warehouses

- Definitions, rationale, architectures, design, implementation, and manipulation; dimensional modelling techniques: design of star and snowflakes schemas, type of fact tables, conversion of relational model into dimensional model.

4. Document-oriented data stores

- Definitions, rationale, architectures, design, implementation, and manipulation; differentiating between document-oriented, key-value pairs, column-oriented and graph types; designing schema-less data structures.

Indicative Syllabus

5. Data and database integration

- Process, challenges, benefits; ETL process for different database types; standards for interoperability and integration including JSON and Web services.

6. Optimisation and performance tuning

- Understanding the reasons for and the implications of poorly performing queries; identifying poorly performing queries and structures; examining how queries are executed in various database technologies; using constraints and indexes; choosing between alternative execution plans; amending queries to perform more appropriately.

7. Security and integrity

- Requirements, e.g. confidentiality, integrity, reliability, storage, and availability; security preserving approaches and integrity preserving approaches; transactions, concurrency control; the BASE model vs. the ACID model for transactions management; polyglot persistence;

8. Emerging database technologies and application areas

- Including but not limited to graph databases, data services, data as a service.

Learning Outcomes

MLO1: Discuss the challenges of, and discriminate between approaches to, database modelling – conceptual, logical, and physical design;

MLO2: Discuss database considerations for data integrity, integration, security, query optimisation, performance tuning and concurrency control in contemporary enterprise applications;

MLO3: Apply techniques for data integrity, security, data optimisation, performance tuning and concurrency control;

MLO4: Understand and apply different approaches to data integration, both semantic and physical;

MLO5: Understand and apply data modelling techniques for logical and physical design to support contemporary enterprise applications;

MLO6: Design, create, and query a data warehouse;

MLO7: Design, create, and query a distributed database;

MLO8: Design, create, and query a document-oriented data store;

MLO9: Critically compare the strengths and limitations of different database technologies used in contemporary enterprise applications;

MLO10: Discuss recent developments and emerging trends in database technologies and their use in contemporary enterprise applications;

Assessment

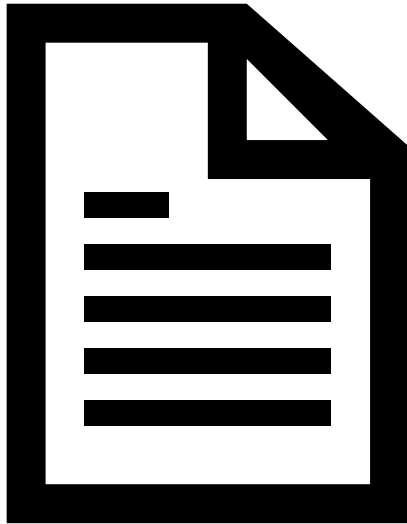
Assessment	Weight	Focus
CA1 – PostgreSQL: Performance & Data Warehousing	20%	Analyse and improve Campus Events workloads; design, implement and query an analytical data warehouse.
CA2 – Modern Data Models	20%	Model and implement new Campus Events requirements using document, key-value/in-memory and vector approaches; justify appropriate technology choices.
CA3 – Distributed Databases	20%	Investigate distributed database techniques including replication and partitioning; experiment with multi-node database behaviour; analyse failure, availability, consistency and distribution trade-offs.

Assessment

	What you will do
1. Complete the task	Develop your Campus Events solution following the specified naming and submission requirements.
2. Test your solution	Run the provided tests to check that your solution is correctly structured and working.
3. Submit your solution	Submit the required SQL, scripts and/or configuration files to Brightspace
4. Verification quiz	Complete a time-based, closed-book quiz based on the practical to demonstrate that you understand and can justify your solution.
5. Automated assessment	Your solution will be executed and checked using additional tests, including tests you have not seen.
6. Follow-up if required	You may be asked to briefly run, modify or explain your solution where clarification is required.

Passing the provided tests does not guarantee full marks. Additional tests will be used during marking.

You are responsible for ensuring that your submitted solution works and that you understand the code and design decisions it contains.



Written Exam 40%

The written examination assesses your ability to **apply, compare and justify** advanced database concepts and approaches.

What will the exam assess?

- Understanding of the **key concepts and principles** covered throughout the module
- Applying database concepts to **new scenarios and requirements**
- Comparing different **data models, database technologies and architectures**
- Analysing **performance, scalability, consistency and availability** requirements
- Interpreting database designs, architectures and query/performance information
- **Justifying** an appropriate database approach for a particular workload

What should you expect?

- Questions will focus on **understanding and application rather than memorising syntax**.
- You may be given a scenario and asked to:
- identify the database requirements → recommend an approach → explain the trade-offs → justify your decision

The practical assessments ask: *Can you build and test it?*

The written examination asks: *Can you analyse a database problem and justify an appropriate solution?*

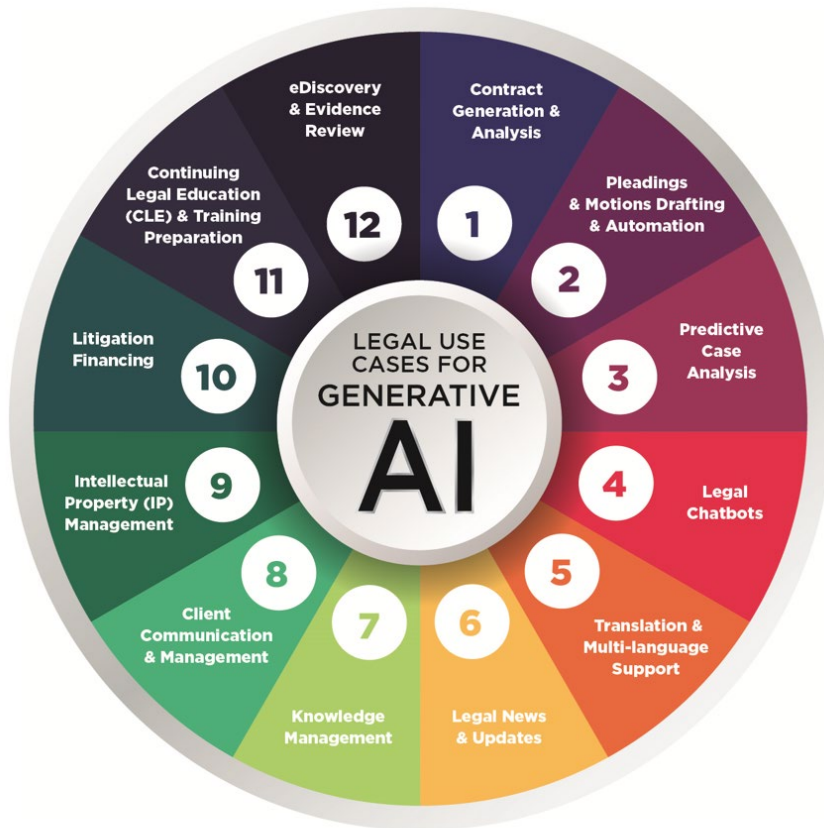
GenAI in CMPU4003

Generative AI can be used as a **learning and development tool**, subject to the requirements specified for each assessment.

You may use GenAI to help you:

- Understand database concepts and terminology
- Explore possible approaches to a problem
- Explain error messages
- Debug or review code
- Explain SQL, Redis commands and configurations
- Compare alternative database designs or architectures

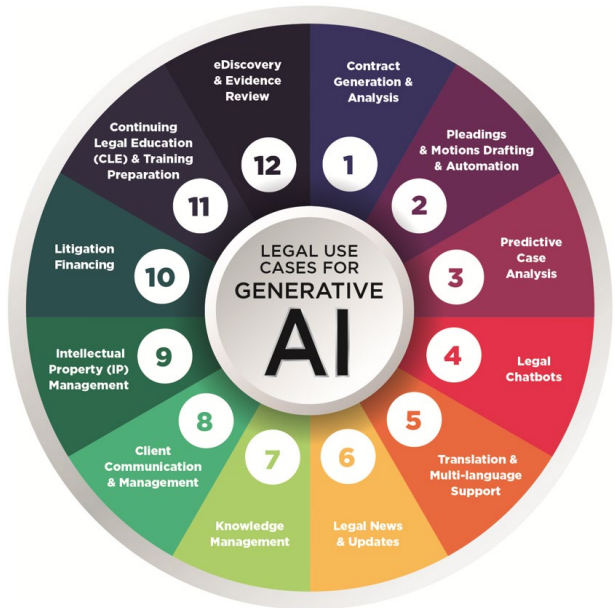
You remain responsible for your work.



Created by: eDis

[This Photo](#) by Unknown Author is licensed under [CC BY](#)

Don't Just Generate — Understand and Verify



This Photo by Unknown Author is licensed under [CC BY](#)

GenAI can produce code that runs but is incorrect, inefficient or inappropriate for the requirements.

For assessed work you must be able to:

- **Run and test** the solution you submit
- Understand the **code and configuration**
- Explain the **design decisions** made
- Interpret the results produced
- **Modify or troubleshoot** your solution if asked
- Justify why your approach is appropriate for the workload

How this will be assessed

- Your submitted code will be executed and tested.
- You will complete an individual quiz based on the practical work.
- You may be asked to demonstrate, modify or explain your solution.

Using GenAI does not transfer responsibility for the solution — if you submit it, you are expected to understand and be able to justify it.

Reading List

Henry F. Korth, S. Sudarshan, Abraham Silberschatz, Professor. (2019), Database System Concepts, 7th. McGraw-Hill Education, p.0, [ISBN: 9780078022159].

Thomas M. Connolly, Carolyn E. Beg. (2015), Database Systems, Addison-Wesley, p.0, [ISBN: 978-0-13-294326-0].